

Analiza leksykalna:

- usuwanie komentarzy
- usuwanie białych znaków
- wydzielanie tokenów
- tworzenie tablicy symboli

Przykładowe błędy:

- niedozwolony znak
- komentarz, który nie został zamknięty
- komentarz, który nie został otwarty
- literał(tekst), który nie został zamknięty

Analiza składniowa:

- sprawdzenie poprawności składniowej
- tworzenie drzewa składniowego programu
- zarządzanie przebiegiem kompilacji (współpraca ze skanerem, analizatorem semantycznym, generatorem kodu i optymalizatorem)

Przykładowe błędy:

- brak średnika
- niesparowane nawiasy
- brak słowa kluczowego

Analiza semantyczna:

- określenie znaczenia programu źródłowego
- wspomaganie analizy składniowej (stosy semantyczne)
- sprawdzanie poprawności semantycznej (akcje semantyczne)
- generowanie kodu pośredniego

Przykładowe błędy:

- niezdefiniowana zmienna / funkcja
- niezgodność typów

Formalny opis języka i gramatyki:

Alfabet V – skończony i niepusty zbiór symboli

Konkatenacja – łączenie kolejnych symboli

Zdanie (słowo) σ nad alfabetem V – każdy, skończony i niepusty ciąg symboli alfabetu V lub symbol pusty ϵ

V^+ - zbiór wszystkich niepustych słów nad V

V^* - zbiór wszystkich słów nad V wraz ze słowem ϵ

Gramatyka formalna:

V_T - skończony zbiór symboli końcowych (terminalnych)

V_N - skończony zbiór zmiennych (symboli nieterminalnych)

Produkcja p – reguła typu (tu se znajdziesz w wykładach bo nie chce się klepac)

Gramatyka formalna G – uporządkowana czwórka (V_T, V_N, S, P) , gdzie S – wyróżniony, początkowy symbol gramatyki; P – zbiór produkcji, określający zbiór formalnie poprawnych zdań w danym języku

Forma zdaniowa σ gramatyki G – każde wyrażenie, dające się wyprowadzić z symbolu początkowego S gramatyki G

Język L wygenerowany przez gramatykę G – zbiór wszystkich form zdaniowych σ , składających się wyłącznie z symboli terminalnych gramatyki

Gramatykę G nazywamy **rozstrzygalną** jeżeli można zbudować algorytm rozstrzygający, czy zdanie σ należy do języka $L(G)$

Język L posiadający powyższą własność nazywamy **obliczalnym**.

Gramatykę nazywamy **niejednoznaczna**, jeżeli istnieje zdanie σ , które można wyprowadzić z symbolu początkowego S gramatyki przynajmniej dwoma różnymi sposobami.

Klasyfikacja gramatyk wg Chomsky'ego:

- Gramatyka typu 0 – **swobodna**

Brak ograniczeń na produkcje gramatyki

- Gramatyka typu 1 – **nieskracająca, kontekstowo zależna**

Prawa strona każdej produkcji musi mieć co najmniej tyle symboli, ile występuje po lewej stronie

- Gramatyka typu 2 – **bezkontekstowa**

Lewa strona produkcji jest zawsze pojedynczym symbolem nieterminalnym.

Gramatyki bezkontekstowe służą do opisywania języków programowania; generują język, dla którego **zawsze** można stworzyć analizator (przy spełnieniu dodatkowych warunków **automatycznie!**)

- Gramatyki typu 3 – regularne

Liniowe – po obydwu stronach produkcji wykorzystuje się nie więcej niż jeden symbol nieterminalny

Prawostronnie liniowe

Prawostronnie rekursywna – po lewej i prawej stronie występuje ten sam symbol nieterminalny

Gramatyki regularne służą do opisu elementów leksykalnych języków (tokenów)

Automaty:

Automat – logiczna maszyna stanów składająca się w ogólności z:

zbioru stanów,

reguł definiujących przejścia pomiędzy stanami,

pamięci,

wejścia i wyjść,

zbioru symboli wejściowych,

zestawem instrukcji wykonywanych przez automat

Automat musi się zawsze znajdować w którymś z dozwolonych stanów (wejściowy, wyjściowy, pośredni). Przejścia pomiędzy stanami odbywają się na skutek wykonania instrukcji automatu:

przeczytanie symbolu terminalnego z wejścia

przeczytanie symbolu nieterminalnego z pamięci

wykonanie innej instrukcji

Funkcje automatów: (uwaga, poniżej skrót myślowy :P)

translatory – wejście/wyjście

analizatory – wejście

generatory – wyjście

Automatem **deterministycznym** nazywamy automat, w którym dla każdego stanu istnieje możliwość przejścia pod wpływem dowolnego symbolu do wyłącznie jednego stanu

Automaty skończone (FA – finie-state automation lub finie-state acceptor):

Typy automatów skończonych:

deterministyczny

niedeterministyczny

niedeterministyczny z ϵ -przejściami

Deterministyczne automaty skończone - DFA :

DFA jest automatem, który dla każdego stanu i dla każdego znaku wejściowego posiada zdefiniowany co najwyżej jeden stan, do którego może nastąpić przejście

Niedeterministyczne automaty skończone – NFA:

NFA jest automatem DFA, w którym zezwolono na określenie więcej niż jednego stanu, do którego może nastąpić przejście ze stanu bieżącego pod wpływem symbolu wejściowego

Skończone automaty niedeterministyczne mają tę samą moc co deterministyczne, gdyż **każdy NFA można przekształcić w odpowiadający mu DFA**

Niedeterministyczne automaty skończone z ϵ -przejściami:

Niedeterministyczne automaty skończone z ϵ -przejściami powstają, jeżeli pozwolimy, by w automacie niedeterministycznym NFA odbywały się przejścia między stanami bez czytania znaku z ciągu wejściowego.

Analizator leksykalny (skaner):

Czynności wykonywane przez analizator leksykalny:

eliminacja komentarzy i białych znaków

obsługa dyrektyw

makroprzetwarzanie

wyodrębnianie tokenów

wykrywanie błędów leksykalnych takich, jak np. użycie niedozwolonych znaków końcowych, brak znaków końca lub otwarcia komentarza, niedokończony literał lub znak

inne czynności takie, jak np. obsługa tablicy symboli i literałów

Analiz składniowa:

Gramatyki bezkontekstowe $XY(k)$:

X – kierunek analizy tekstu wejściowego:

L – od lewej strony

R – od prawej strony

Y – kolejność tworzenia symboli nieterminalnych w drzewie:

L – począwszy od korzenia w kierunku liści

R – począwszy od liści w kierunku korzenia

k – maksymalna liczba symboli terminalnych, które należy „podejrzeć” w przód, aby podjąć jednoznaczną decyzję o wyborze produkcji (tylko dla analizatora bez nawrotów)

Analizatory LR:

Własność ważnego prefiksu (valid prefix property) – wykrywanie błędów w najwcześniejszej możliwej chwili

Prostą frazę (simple phrase) formy zdaniowej $\sigma = \varphi_1\beta\varphi_2$ dla pewnego symbolu nieterminalnego $A \in V_N$ nazywamy ciąg $\beta \in V^*$, jeżeli $S \Rightarrow^* \varphi_1 A \varphi_2 \cap A \rightarrow \beta \in P$ (po chuju to wygląda, nie wiadomo o co chodzi, ale tak jest - przyp. autora)

Uchwyt (handle) – formy zdaniowej jest jej położoną najbardziej na lewo prostą frazę

Prefiksem wykonalnym (viable prefix) lub **prefiksem** formy zdaniowej $\varphi\beta t$, gdzie β jest uchwytem, a $t \in V_T^*$, nazywamy dowolny początkowy podciąg łańcucha $\varphi\beta$

Tworzenie drzewa wyvodu:

Shift x – czytanie symbolu terminalnego z ciągu wejściowego. Symbol ten staje się liściem drzewa wyvodu umieszczonym na prawo od poprzednich liści. Nowym stanem staje się x. Jest on umieszczany na wierzchołku stosu stanów.

accept – przeczytany łańcuch jest akceptowany, a proces analizy zostaje zakończony

error – przeczytany łańcuch nie jest akceptowany. Proces analizy może zostać zakończony lub też może nastąpić próba wznowienia go przy ominięciu pewnej części łańcucha wejściowego (error recovery). W obu przypadkach sygnalizowany jest błąd.

reduce y – odpowiada stanowi redukcji automatu deterministycznego. y oznacza numer produkcji $N \Rightarrow \alpha$, którą należy zastosować. Tworzony jest węzeł drzewa o nazwie N, zgodnie z nazwą symbolu nieterminalnego występującego po lewej stronie produkcji. do tego wierzchołka dołączanych jest $|\alpha|$

korzeni dotychczas zbudowanych poddrzew (od prawej strony). Ze stosu zdejmowanych jest również $|\alpha|$ stanów. Nowy stan pobiera się z tablicy akcji z wiersza określonego przez stan na wierzchołku stosu i z kolumny określonej przez N. Nowy stan jest odkładany na stos stanów.

O analizatorach LR i tworzeniu tablicy akcji jest trochę za dużo więc odsyłam do wykładów.

Analizatory LL

Podejście siłowe top-down:

1. Stosowana jest pierwsza produkcja dla symbolu początkowego gramatyki
2. W nowo rozwiniętym ciągu, stosowana jest pierwsza produkcja dla pierwszego symbolu nieterminalnego (z lewej strony)
3. Operacja ta jest powtarzana do chwili, gdy:
 - w rozwiniętym ciągu nie pozostał żaden symbol nieterminalny (zdanie zostaje zaakceptowane)
 - w wyniku rozwinięcia ciągu powstała niezgodność początkowej sekwencji symboli nieterminalnych z elementami analizowanego zdania. W takim wypadku następuje nawrót, polegający na cofnięciu ostatnio zastosowanej produkcji i zastosowaniu następnej możliwej. Jeżeli na bezpośrednio wyższym poziomie wyczerpane zostały wszystkie możliwe produkcje, to następuje nawrót na kolejny, wyższy poziom. Oczywiście dotarcie z powrotem do symbolu początkowego oznacza, że dany ciąg wejściowy nie należy do akceptowanego języka.

Wady podejścia siłowego:

bardzo duża złożoność obliczeniowa, nawet dla prostych gramatyk

powolność działania związana z koniecznością wykorzystania mechanizmu rekursywnego wołania procedur

znikoma zdolność do eliminacji błędów kompilacji (error recovery) związana głównie z niemożnością dokładnego wskazania miejsca wystąpienia błędu

konieczność skomplikowanego zarządzania blokami pamięci dla generowanego kodu programu, które muszą być wymazywane przy każdym nawrocie

Analizatory top-down z ograniczonymi nawrotami

Metoda rekursywnego zagłębiania (RDP – Recursive-Descent Parsers)

Proste gramatyki LL(1)

Prostą gramatyką LL(1) (SLL(1) – Simple LL) nazywamy gramatykę bezkontekstową bez reguł pustych taką, że dla każdego symbolu nieterminalnego $A \in V_N$, produkuje rozszerzające się z tego symbolu rozpoczynają się wszystkie różnymi symbolami terminalnymi

Przenośność (portability) oraz adaptowalność (adaptability):

O programie mówimy, że jest przenaszalny, jeżeli może być on przeniesiony na inną maszynę ze względną łatwością.

Program jest adaptowalny, jeżeli można łatwo dostosować go do różnych wymagań użytkownika i wymagań systemowych.

Optymalizacja kodu:

Cele:

szybsze działanie i/lub mniejsza objętość kodu

- w tej samej konfiguracji sprzęt + oprogramowanie
- w kolejnych konfiguracjach sprzętu i oprogramowania

szybsze tworzenie oprogramowania

oprogramowanie bezpieczniejsze, uniwersalne, ...

optymalny dostęp do reglamentowanych zasobów systemu

Strategie:

właściwe kodowanie

ręczna optymalizacja kodu

preprocesor

kompilator

linker

interpreter

Prawo Proebstiga : „Technologia kompilatorów podwaja prędkość programów co 18 lat (???)”

Wybrane techniki optymalizacji:

optymalizacja słownikowa (w okienkach)(peephole optimization)

optymalizacja siłowa

optymalizacja wywołań funkcji

optymalizacja alokacji rejestrów (cross-call register allocation)

wstawianie funkcji (procedure inlining)

klonowanie funkcji (procedure cloning)